```python
"""Supplied signal generation, WAV I/O, metrics and QPSK receiver. No student DSP answers."""
from pathlib import Path
import numpy as np
from scipy import signal
from scipy.io import wavfile

FS = 16000

def write_wav(path, x, fs=FS, scale=1.0):
    """Float32 WAV. I/Q means channel 0=I, channel 1=Q; no independent normalization."""
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    a = np.column_stack((x.real, x.imag)) if np.iscomplexobj(x) else x
    if np.max(np.abs(a * scale)) >= 1:
        raise ValueError('Use one shared scale for all comparison files to prevent clipping')
    wavfile.write(path, fs, (a * scale).astype(np.float32))

def read_wav(path, iq=False, target_fs=FS):
    fs, a = wavfile.read(path)
    if np.issubdtype(a.dtype, np.integer):
        if a.dtype == np.uint8:
            a = (a.astype(float) - 128) / 128
        else:
            a = a.astype(float) / (2 ** (np.iinfo(a.dtype).bits - 1))
    else:
        a = a.astype(float)
    if iq:
        if a.ndim != 2 or a.shape[1] != 2:
            raise ValueError('I/Q requires exactly two channels')
        a = a[:, 0] + 1j * a[:, 1]
    elif a.ndim == 2:
        a = a.mean(axis=1)
    if fs != target_fs:
        from math import gcd
        g = gcd(fs, target_fs)
        a = signal.resample_poly(a, target_fs // g, fs // g)
    return target_fs, a

def relerr(x, y):
    return float(np.linalg.norm(y-x) / np.linalg.norm(x))

def snr(clean, estimate):
    return float(10*np.log10(np.sum(np.abs(clean)**2) /
                             max(np.sum(np.abs(estimate-clean)**2), 1e-30)))

def rrc(beta=.35, sps=16, span=10):
    """Unit-energy root-raised-cosine pulse; span in symbols, odd number of taps."""
    t = np.arange(-span*sps//2, span*sps//2+1)/sps
    p = np.empty_like(t)
    for i, v in enumerate(t):
        if abs(v) < 1e-12:
            p[i] = 1 + beta*(4/np.pi-1)
        elif abs(abs(v)-1/(4*beta)) < 1e-12:
            p[i] = beta/np.sqrt(2)*((1+2/np.pi)*np.sin(np.pi/(4*beta)) +
                                    (1-2/np.pi)*np.cos(np.pi/(4*beta)))
        else:
            p[i] = (np.sin(np.pi*v*(1-beta)) + 4*beta*v*np.cos(np.pi*v*(1+beta))) / (np.pi*v*(1-(4*beta*v)**2))
    return p / np.linalg.norm(p)

def fixtures(folder):
    folder = Path(folder); folder.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(2735)

```

```
    t = np.arange(4*FS)/FS
    # Low-frequency desired audio and abrupt attacks expose smearing and leakage.
    melody = np.zeros_like(t)
    notes = [220, 277.18, 329.63, 440, 392, 329.63, 277.18, 220]
    for i, f in enumerate(notes):
        idx = (t >= .5*i) & (t < .5*(i+1)); u = t[idx]-.5*i
        env = (1-np.exp(-u/.008))*np.exp(-u/0.3)
        melody[idx] = env*(np.sin(2*np.pi*f*u)+.3*np.sin(4*np.pi*f*u))
    melody *= .32
    jammer = .5*np.cos(2*np.pi*(1800*t + .5*300*t*t)+.4)
    mixture = melody + jammer
    probe = .25*np.cos(2*np.pi*1000*t)+.25*np.cos(2*np.pi*1080*t)
    probe += .25*signal.chirp(t, 500, 4, 3500)
    transient = np.zeros_like(t)
    transient[int(.9*FS):int(.9*FS)+80] = .28*signal.windows.hann(80)
    probe += transient/.7
    probe *= .7
    for name, a in [('melody', melody), ('jammer', jammer), ('jammed', mixture),
('tf_probe', probe)]:
        write_wav(folder / (name+'.wav'), a)
    # One finite frame for leakage; the off-bin tone reveals the window sidelobes.
    n = np.arange(1024)
    leakage = .65*np.cos(2*np.pi*1003*n/FS)+.015*np.cos(2*np.pi*1140*n/FS)
    write_wav(folder/'leakage.wav', leakage)
    bits = rng.integers(0, 2, (2048, 2))
    symbols = ((1-2*bits[:, 0])+1j*(1-2*bits[:, 1]))/np.sqrt(2)
    p = rrc(); up = np.zeros(len(symbols)*16, complex); up[::16] = symbols
    tx = np.convolve(up, p)
    write_wav(folder/'qpsk_iq.wav', tx)
    np.savez(folder/'truth.npz', bits=bits, symbols=symbols, pulse=p, tx=tx,
             melody=melody, jammer=jammer, mixture=mixture, t=t)
    return dict(bits=bits, symbols=symbols, pulse=p, tx=tx, melody=melody,
                jammer=jammer, mixture=mixture, probe=probe, transient=transient, t=t)

def receive(x, symbols, bits, pulse):
    """Known timing, no carrier recovery or per-symbol correction. Transients excluded."""
    mf = np.convolve(x, pulse.conj()[::-1])
    z = mf[(len(pulse)-1)+16*np.arange(len(symbols))]
    keep = slice(10, -10); z = z[keep]; ref = symbols[keep]; b = bits[keep]
    evm = 100*np.linalg.norm(z-ref)/np.linalg.norm(ref)
    decisions = np.column_stack((z.real < 0, z.imag < 0))
    ber = np.mean(decisions != b)
    return dict(evm_percent=float(evm), ber=float(ber)), z

```